

Algorithmization and Programming of Solutions

Part I

TOPIC #6

AN INTRODUCTION TO DATA STRUCTURES: ARRAYS IN PROGRAMMING LANGUAGE C

RTU LECTURER OLGA YAKOVLEVA, MG.SC.ING

Table of contents

1. An introduction to data structures:

- a concept of a data structure
- a concept of an array
- arrays in programming language C
- examples of array processing
- pointers

1. An introduction to data structures

A CONCEPT OF A DATA STRUCTURE, A CONCEPT OF AN ARRAY,
ARRAYS IN PROGRAMMING LANGUAGE C, EXAMPLES OF ARRAY
PROCESSING, POINTERS

A concept of a data structure

A data structure is:

- interrelationship among data elements that determine how data is recorded, manipulated, stored, and presented by a database (<http://www.businessdictionary.com/definition/data-structure.html>)
- any method of organizing a collection of data to allow it to be manipulated effectively (*The Free On-line Dictionary of Computing*, <http://foldoc.org>)

Data structure consists of **elements** and **connections** between these elements, where elements are simple data (e.g., *integer number*) and/or structured data (e.g., *string*)

A concept of an array (1/2)

An array is:

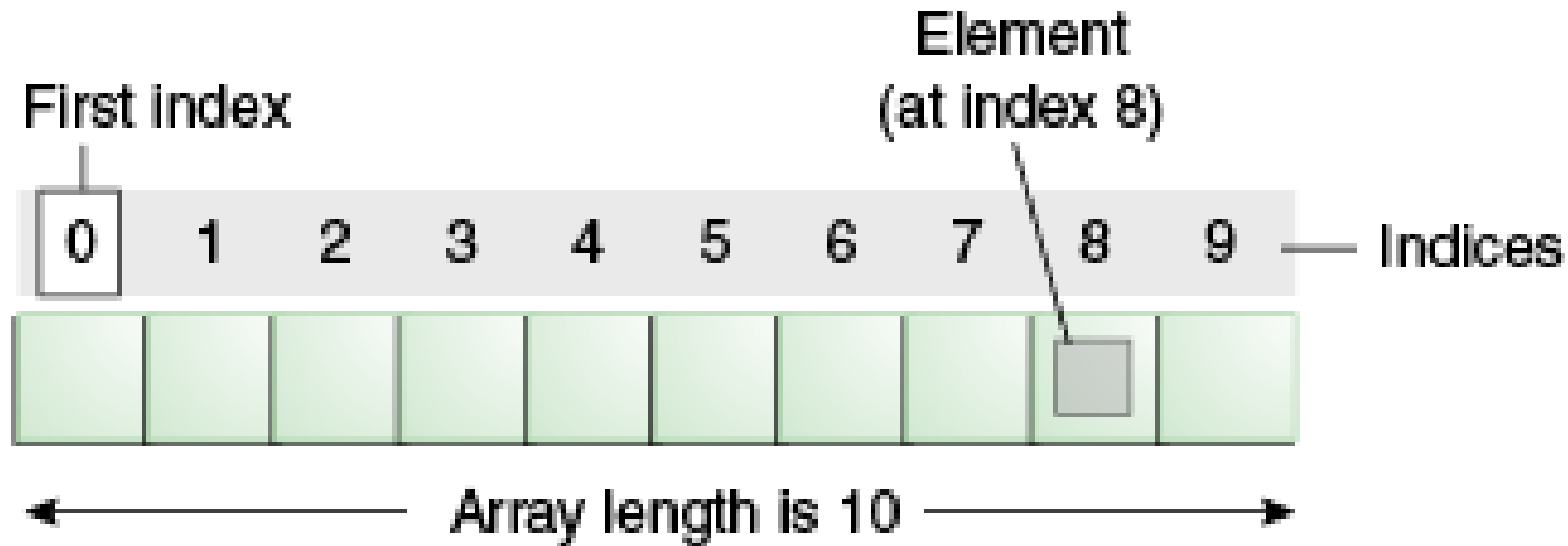
- a list of data values, all of the same type, any element of which can be referenced by an expression consisting of an array name followed by an indexing expression (*Microsoft Terminology 2011. Entry from the Microsoft Language Portal*)
- a fundamental data structure that supports the ability to access an element at an arbitrary position [a random access method] (*G.Matisons. Datu struktūras. Lekciju konspekts. – Rīga. 2003*)

The elements of an array are stored sequentially in a computer memory and each element has a specified position that is called an index

A concept of an array (2/2)

An original source of the picture: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html>

Usually, indices of array elements **start with 0** and are assigned by default while declaring an array



Types of arrays

An array can be **static** (i.e., size (number of elements) of an array **can't be changed** during an execution of a program) or **dynamic** (i.e., size (number of elements) of an array **can be change** during an execution of a program)

By a number of dimensions arrays fall into:

- **single dimensional:**
 - *one-dimensional* (elements are situated in one row)
- **multi-dimensional:**
 - *two-dimensional* (elements are situated in several rows, i.e., in a table)
 - *three-dimensional* (elements are situated in several tables, i.e., in a cube)
 - etc.

Dimensions of an array

An original source of the picture: <https://book.mql4.com/ru/variables/arrays>

A dimension of an array is a number of indices, which is needed to pick an element of an array

*index = sequential
number*

a)

	0	1	2	3	4	5	6	7	8	9
	7	3	5	12	34	2	5	56	-7	4

*index = row
sequential number
+ column
sequential number*

b)

	0	1	2	3	4	5	6	7	8	9
0	7	3	5	12	34	2	5	56	13	4
1	0	-8	-4	10	54	33	-4	98	12	12
2	34	12	-6	6	94	52	1	0	10	10
3	2	3	4	7	4	55	12	12	34	77
4	34	12	-6	0	-6	1	1	0	54	3
5	14	3	-7	-4	88	-3	12	67	24	-2
6	3	-2	34	-5	6	-5	3	30	8	11
7	44	5	14	28	13	-6	53	23	71	5
8	12	3	39	68	38	0	1	58	0	22
9	33	23	-7	21	5	2	67	34	8	9

c)

		2	0	1	2	3	4	5	6	7	8	9	
1			2	-4	22	-4	17	-4	61	67	12	34	0
0			7	3	5	12	34	2	5	56	13	4	1
	33	-2	7	-5	23		54	33	-4	98	12	12	2
	2	3	4	7	4		94	52	1	0	10	10	3
	34	12	-6	6	94		4	55	12	12	34	77	4
	2	3	4	7	4	55	6	77	1	0	54	3	5
	34	12	-6	0	-6	77	1	0	1	1	24	-2	6
	14	3	-7	-4	88	35	12	67	-3	12		11	7
	3	-2	34	-5	6	14	3	30	-5	3		5	8
	44	-5	14	-6	13	4	53	23	-6	53		22	9
	12	3	39	-0	38	19	1	58	0	1		9	
	33	23	-7	21	5	2	67	34	2	67			

*index = 1st dimension
sequential number +
2nd dimension
sequential number +
... + nth dimension
sequential number*

Arrays in programming language C

Declaration of an one-dimensional array

Declaration:

- *General form*

<Data Type> <Name> [<Length>];

- *Example*

```
int first_array[5]; // an empty array of  
integers consisting of 5 elements, indices  
start with 0
```

```
float second_array[3]; // an empty array of  
float numbers consisting of 3 elements,  
indices start with 0
```

Arrays in programming language C

Initialization of an one-dimensional array

Initialization:

- *General form*

`<Name>[<Index>] = <Value>;`

- *Example*

`first_array[0] = -14; // -14 is assigned to the
1st array element`

`first_array[1] = 2; // 2 is assigned to the 2nd
array element`

`int i = 0;`

`second_array[i] = 6; // 6 is assigned to the 1st
array element`

Arrays in programming language C

Declaration and initialization of an one-dimensional array (1/2)

Declaration and initialization:

- *General form*

```
<Data Type> <Name>[<Length>] = {<value1>,  
<value2>, ...};
```

- *Example*

```
// an array of integers with values 2, 4, 6,  
8 and 10
```

```
int first_array[5] = {2, 4, 6, 8, 10};
```

```
// or
```

```
int first_array[] = {2, 4, 6, 8, 10};
```

Arrays in programming language C

Declaration and initialization of an one-dimensional array (2/2)

Declaration and initialization:

- *Example*

```
// an array of float numbers consisting of 3  
elements with values 1.0, 2.1 and 3.5
```

```
float second_array[] = {1.0, 2.1, 3.5};
```

```
// or
```

```
// an array of float numbers consisting of 3  
elements with values 1.0 and 2.1 and one  
empty element
```

```
float second_array[3] = {1.0, 2.1};
```

Arrays in programming language C

Declaration of a two-dimensional array

Declaration:

- *General form*

```
<Data Type> <Name>[<Size of 1st dimension>]  
[Size of 2nd dimension];
```

- *Example*

```
char array2[2][3]; // an empty array of  
characters consisting of 6 elements (2 rows  
and 3 columns)
```

Arrays in programming language C

Initialization of a two-dimensional array

Initialization:

- *General form*

`<Name> [<Index>] [<Index>] = <Value>;`

- *Example*

`array2[0][0] = 'a'; // 'a' is assigned to the 1st
array element`

`array2[1][2] = 'z'; // 'z' is assigned to the
last array element`

`int i = 1; j = 0;`

`array2[i][j] = 'd'; // 'd' is assigned to the 1st
element of the 2nd row`

Arrays in programming language C

Declaration and initialization of a two-dimensional array

Declaration and initialization:

- *General form*

```
<Data Type> <Name>[<Size1>][<Size2>] =  
{ {<value1_1>, <value1_2>, ...},  
  {<value2_1>, <value2_2>, ...}, ...};
```

- *Example*

```
// an array with values 2, 4, 6, 8, 10 and 12  
int arr[2][3] = {{2, 4, 6}, {8, 10, 12}};  
// or  
int arr[][3] = {{2, 4, 6}, {8, 10, 12}};  
// or  
int arr[2][3] = {2, 4, 6, 8, 10, 12};
```

Filling an one-dimensional array with random numbers (1/2)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main () {
    int i, mas[5];
    srand(time(NULL)); // initialize a starting
                        // point for generating of
                        // random numbers
```

Filling an one-dimensional array with random numbers (2/2)

```
for (i=0; i<5; i++) {  
    mas[i]=rand(); // generates random numbers  
                    in a range [0..RAND_MAX]  
    printf("%d\t", mas[i]);  
}  
return 0;  
}
```

Filling an one-dimensional array with user inputted numbers (1/2)

```
#include <stdio.h>
```

```
void main() {
```

```
    int a[3], i;
```

```
    printf("\n\t Enter three numbers: ");
```

```
    for(i=0; i<3; i++) {
```

```
        scanf("%d", &a[i]);
```

```
    }
```

Filling an one-dimensional array with user inputted numbers (2/2)

```
printf("\n\n\t Numbers are: ");  
for(i=0; i<3; i++) {  
    printf("\t %d", a[i]);  
}  
return 0;  
}
```

Example of an one-dimensional array processing (1/4)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main () {
    int i, temp, mas[10];
    bool swap;
    srand(time(NULL));
```

Example of an one-dimensional array processing (2/4)

```
for (i=0; i<10; i++) {  
    mas[i]=rand()%30+1;  
    printf("%d\t", mas[i]);  
}  
  
do {  
    swap=false;
```

Example of an one-dimensional array processing (3/4)

```
for (i=0; i<9; i++) {  
    if (mas[i]%2==0 && mas[i+1]%2!=0) {  
        temp=mas[i];  
        mas[i]=mas[i+1];  
        mas[i+1]=temp;  
        swap=true;  
    }  
}
```

Example of an one-dimensional array processing (4/4)

```
    }  
} while (swap == true);  
for (i=0; i<10; i++)  
    printf("%d\t", mas[i]);  
return 0;  
}
```

Filling a two-dimensional array with random numbers (1/2)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main () {
    int i, j, mas[5][4];
    srand(time(NULL)); // initialize a starting
                        // point for generating of
                        // random numbers
```

Filling a two-dimensional array with random numbers (2/2)

```
for (i=0; i<5; i++) {  
    for (j=0; j<4; j++) {  
        // assigns values in a range [-15..14]  
        mas[i][j]=rand()%30-15;  
        printf("%5d", mas[i][j]);  
    } printf("\n");  
}  
return 0;  
}
```

Concept of a pointer, static and dynamic memory allocation

Pointer is a programming language data type whose value refers directly to (or "points to") another value stored elsewhere in a computer memory using its address

Pointer variable contains a memory address of some data

Static memory allocation refers to the process of allocating memory at compile-time before the associated program is executed, unlike **dynamic memory allocation** refers to the process of allocating memory as required at run-time

Static and dynamic variables

```
int i; // an integer type variable
int *p; // a pointer type variable
```

...

```
p = malloc(sizeof(int));
i = 5;
```

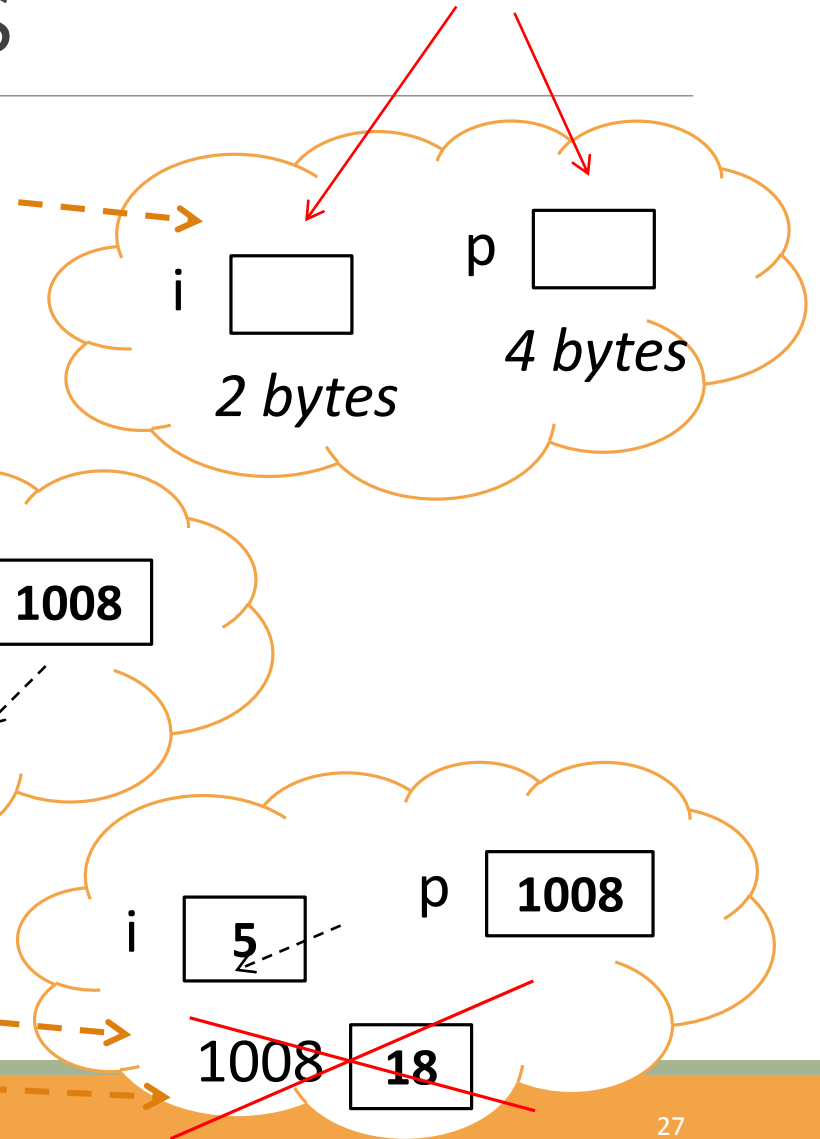
...

```
*p = 18;
```

```
free(p);
```

static
variables

dynamic variable



A name of an array as a pointer

A name of an array can be used as a pointer, e.g.:

```
int a[5] = {1, 2, 3, 4, 5};
```

- `*a` is the same as `a[0]`
- `*(a+1)` is the same as `a[1]`
- `*(a+k)` is the same as `a[k]`
- `*(a+3) += *a - *(a+1) / *(a+2);`
- `a[2] == *(a+2) == *(2+a) == 2[a];`
- `int *p = a;`
`*p` is the same as `a[0]` and the same as `p[0];` // etc.